

BELLS UNIVERSITY EXPENSE SPLITTER
BY
COMPUTER SCIENCE (300 Level)
GROUP 29



COLLEGE OF NATURAL AND APPLIED SCIENCES
BELLS UNIVERSITY OF TECHNOLOGY - NEW HORIZONS

January 2026
PYTHON PROGRAMMING
(BUT-ICT323)

SUBMITTED TO
AYUBA MUHAMMAD

List of Team Members (Group 29)

Ayinde Muktar Oluwasegun	2023/12672
Ayoade Abdulfattah	2024/13788
Bakare Chukwuemeka	2023/12333
Goodluck Williams	2023/12468
Friday Ekeminiabasi	2023/12509

Table of Contents

Abstract.....	5
Chapter 1: Introduction	6
1.1 Background of the Study	6
1.2 Problem Statement	6
1.3 Aim and Objectives	6
1.4 Significance of the Project.....	6
1.5 Scope and Limitations	7
1.6 Organization of the Study.....	7
Chapter 2: Literature Review / Related Work.....	8
2.1 Review of Existing Similar Applications	8
2.2 Strengths and Weaknesses of Existing Systems	8
2.3 Python Libraries Used in Similar Projects.....	8
2.4 Identified Research Gap.....	8
Chapter 3: Methodology	9
3.1 System Architecture	9
3.2 Tools and Libraries Used.....	10
3.3 GUI Design	10
3.4 Algorithm and Flowchart	10
3.5 Database and File Structure	11
3.6 Development Workflow.....	12
Chapter 4: Implementation and Results	12
4.1 Application Screenshots	12
4.2 Sample Input and Output	15
4.3 Key Code Snippets.....	15
4.4 Performance Evaluation	18
4.5 User Testing Results.....	18
Chapter 5: Conclusion and Recommendations	19
5.1 Summary of Achievements.....	19
5.2 Conclusion	19
5.3 Challenges and Lessons Learned	19
5.4 Future Improvements.....	19
5.5 Contribution and Usefulness	19

Appendices	21
Appendix A: Project Dependencies (requirements.txt).....	21
Appendix B: User Manual.....	22
Appendix C: GitHub Repository	24

Abstract

Group expenses often lead to confusion and disputes when tracking who paid what and how to fairly settle balances, especially in offline scenarios like student trips or shared accommodations where internet-dependent mobile apps fail. This project addresses this challenge by developing an offline desktop application called "Bells University Expense Splitter" using Python and PySide6.

The aim was to create a user-friendly graphical interface for managing group expenses, with accurate real-time calculations, balance tracking, optimal settlement suggestions, and PDF report generation, all without requiring internet connectivity. Specific objectives included designing an intuitive GUI inspired by modern mobile apps, implementing local SQLite persistence, performing precise expense splitting (equal shares with exclusions), generating minimal-transaction settlements using a heap-based algorithm, supporting theme switching (light/dark/system), and packaging the application for easy use.

The methodology employed a layered architecture: QML for responsive UI, Python for business logic and calculations, SQLite via QSql for data storage, and ReportLab for PDF exports. Development followed an iterative workflow with Git version control on GitHub.

The resulting application allows users to create trips, add participants and expenses (with optional exclusions or personal items), view instant balances, receive automated payment recommendations to minimize transactions, and export summaries. User testing with five students confirmed intuitive navigation, accurate computations, and high satisfaction with features like dark mode and PDF sharing.

This project demonstrates Python's capability for building polished, standalone desktop tools and provides a practical, free solution for fair expense management among groups. Future enhancements could include unequal splits and mobile compatibility.

Chapter 1: Introduction

1.1 Background of the Study

Python belongs to the most used programming languages because its syntax is readable, its libraries are extensive and it adapts to many tasks. It is particularly useful for desktop programs that provide a graphical interface - such programs are easier to use than command line tools. When students travel together, roommates share a flat or a family takes a trip, they often lose track of who paid for what plus how to share the cost fairly. A desktop application with a clear window, buttons and numbers on the screen turns the arithmetic into a simple visual task. We therefore wrote an Expense Splitter in Python with the PySide6 toolkit - the program runs on the computer without an internet connection but also presents a modern window while it performs every calculation locally.

1.2 Problem Statement

Group expenses become tangled fast - one participant pays for fuel, another buys food, a third pays for drinks and soon no one remembers the exact amounts or who should reimburse whom. Common phone apps help but they demand accounts as well as steady internet, conditions that are unreliable on road trips, at campus events or in areas with poor network connectivity. We required a plain, offline desktop utility that lets any group type in expenses, watch the balance update at once and receive plain instructions on the simplest way to settle debts without quarrels.

1.3 Aim and Objectives

General Objective: To design and develop a fully functional Python desktop application with GUI, data processing, and visualization for managing and splitting group expenses.

The specific objectives are:

1. To create an intuitive and user-friendly graphical user interface.
2. To implement file handling and data persistence using a local database.
3. To perform accurate expense calculations and balance analysis.
4. To generate clear summaries and settlement suggestions for users.
5. To package the application with proper error handling and usability features.
6. To deploy the application on GitHub with complete documentation.

1.4 Significance of the Project

The program deals with a routine hassle - people who share costs for group work, travel, parties or joint orders no longer need to quarrel over who owes what. It divides every bill fairly and removes the tension that money disputes create. As an academic exercise it also

shows that one developer can handle a desktop interface in Python, link it to a database, write sound algorithms plus work in a team. The finished product demonstrates that Python alone yields a clean, commercial quality desktop tool without web frameworks.

1.5 Scope and Limitations

Users open a trip or group, list every payer and enter each expense. The entry can skip certain people or be labelled personal. Balances update instantly on screen. With one click the program proposes the shortest chain of payments to settle the books. Dark or system themes are available but also a pdf report can be exported. All records reside in a local SQLite file - the app runs without an internet link.

At present the split is always equal - percentage or custom shares are not but coded. The release runs only on desktop. Cloud sync, mobile editions and richer split rules remain options for later releases.

1.6 Organization of the Study

This report has five chapters. Chapter One covers the introduction. Chapter Two reviews related work. Chapter Three explains the methodology. Chapter Four presents implementation and results. Chapter Five concludes with recommendations.

Chapter 2: Literature Review / Related Work

2.1 Review of Existing Similar Applications

Among the best-known phone and web programs are:

- **Splitwise**, which serves large groups and handles unequal shares, receipts and pay-back records.
- **Plates** by Splitwise, which limits itself to restaurant tabs.
- **Tricount**, which works without a web link once installed.
- **Settle Up**, which runs on multiple systems while offline.

Programs that live only on a computer are scarce. GitHub holds a few Tkinter finance tools for one person but hardly any that split costs for a crowd. Many “desktop” copies of Splitwise are no more than browser frames served through WebCatalog.

2.2 Strengths and Weaknesses of Existing Systems

The programs listed above add, divide but also display sums quickly and their screens are easy to read. But most of them demand a live web link, cap the number of groups or transactions as well as hide the way money finally moves. Some burden the user with options that a simple trip with friends does not need - others refuse to run at all without a connection, a clear drawback on the road.

2.3 Python Libraries Used in Similar Projects

Research into similar applications identified several commonly used Python libraries. For GUI development, Tkinter is popular for simple applications, while PyQt and PySide are preferred for projects requiring advanced features and polished interfaces. SQLite dominates local data storage due to its lightweight design and lack of server requirements. Data visualization typically relies on Matplotlib, with some projects using Seaborn or Plotly for interactive charts. ReportLab serves as the standard tool for PDF generation.

Some newer projects experiment with Flet or Streamlit, but these frameworks are primarily designed for web-apps.

2.4 Identified Research Gap

Most existing expense-sharing applications are web-based. They require users to create online accounts and maintain internet connectivity for data synchronization across devices. Options for lightweight, fully offline tools with modern interfaces are scarce. This project addresses that gap by providing a standalone Python desktop application focused on simplicity, transparency, and complete offline functionality.

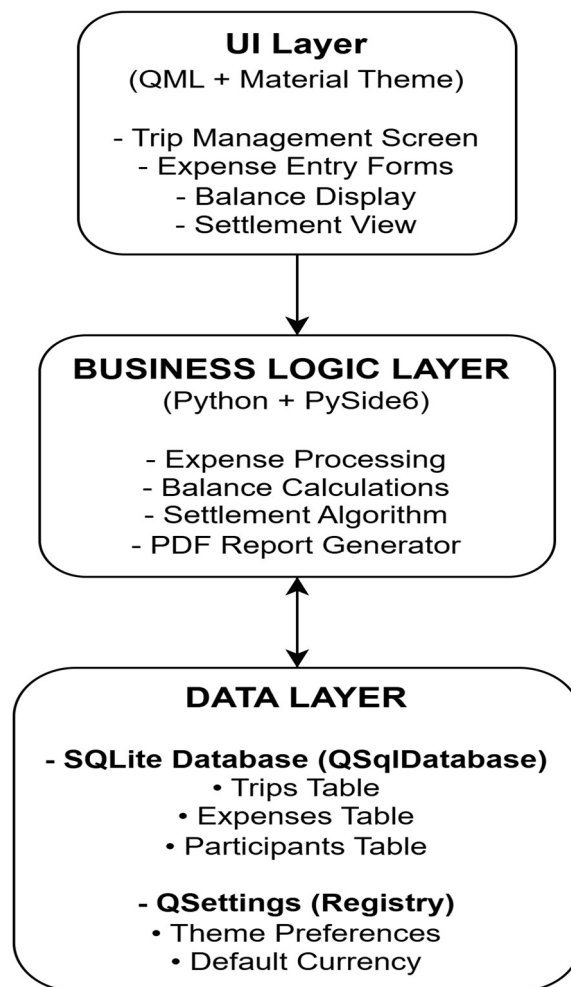
Chapter 3: Methodology

3.1 System Architecture

The app uses a layered architecture:

- **UI Layer:** QML for responsive UI and views.
- **Logic Layer:** Python/PySide6 handles expense processing, balance calculations, settlement algorithm, and PDF generation.
- **Data Layer:**
 - SQLite (via QSqlDatabase) for trips/expenses/participants.
 - QSettings (registry) for app preferences like global currency and theme.

Data flows from user input → backend calculations → GUI updates.



3.2 Tools and Libraries Used

- **Python (3.12+)**: Core programming language used for application development.
- **PySide6 (6.10.1)**: Provides access to the Qt framework for GUI development.
- **QML**: Used to design a modern and responsive user interface.
- **SQLite (via QtSql)**: Accessed through QSqlDatabase, used for local data persistence.
- **ReportLab**: Used for generating PDF documents for summaries.
- **Git/GitHub**: Used for version control and team collaboration.

3.3 GUI Design

The application's interface was designed with inspiration from "Notepad" apps (mobile and desktop):

- **Homepage**: Displays trip cards with a search bar and sorting menu.
- **Trip Details**: Uses tabs to organize for *Expenses*, *Participants (and their balances)*, and *Settlements*.
- **Actions**: Employs a Floating Action Button for adding trips on the homepage and standard buttons for other actions.
- **Themes**: Supports Light, Dark, and System themes.

The interface layout was initially planned using physical sketches on paper.

3.4 Algorithm and Flowchart

Settlement Algorithm:

1. Calculate the net balance for each participant (total paid minus total owed).
2. Separate participants into debtors (negative balance) and creditors (positive balance).
3. Sort debtors in descending order by amount owed.
4. Sort creditors in ascending order by amount to receive.
5. Iteratively match the largest debtor with the largest creditor until all balances are settled, minimizing the number of payments.

Application Flow:

1. User inputs expense details.
2. Expense is saved to database.
3. Participant balances are updated.
4. The interface refreshes to reflect the updated expense list and balances.
5. The settlement view updates to show the optimal payment plan.

The application flow is illustrated in *Fig. 3.1*. It demonstrates how user input is processed, balances updated, and settlement suggestions generated.

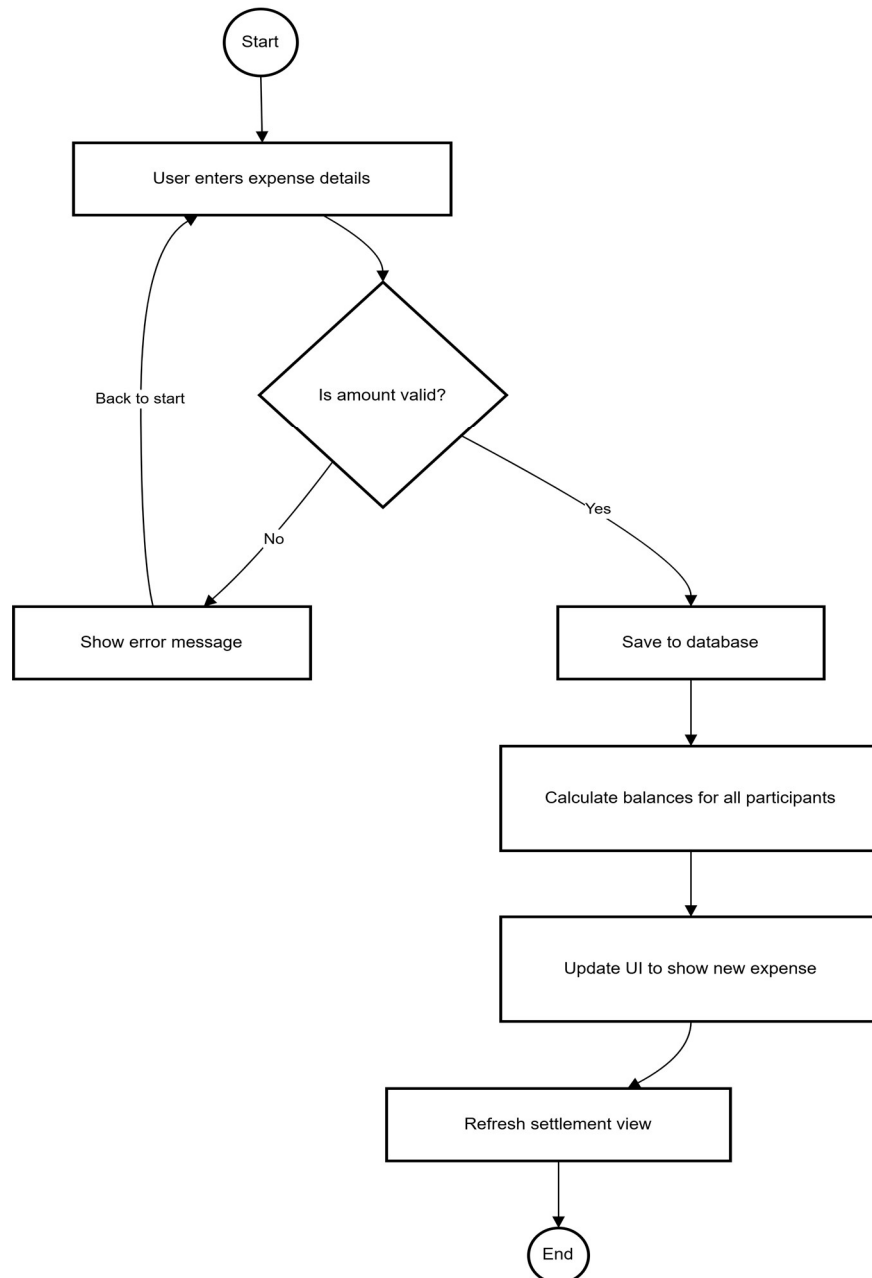


Fig. 3.1: Application Flow

3.5 Database and File Structure

All application data is stored in an SQLite database located in the user's AppData/Roaming directory for persistence. User preferences, such as interface theme and currency, are saved in the system registry via QSettings. When a PDF summary is generated, the file is exported directly to the user's Documents folder for easy access.

3.6 Development Workflow

Version control was managed using Git, with the repository hosted on GitHub to facilitate team collaboration and change tracking. The development process followed these main stages:

- **Planning:** Defined and prioritized core features.
- **Implementation:** Developed features sequentially, with testing after each addition.
- **Testing:** Conducted thorough testing of major features upon completion.
- **Refactoring:** Performed regular code reviews and optimizations to maintain code quality and performance.
- **Documentation:** Maintained consistent code comments and documentation throughout development to ensure clarity and maintainability.

Chapter 4: Implementation and Results

4.1 Application Screenshots

The images illustrate the main trip list, detailed trip pages, views for expenses and participants, settlement recommendations, data entry dialogs, and both light and dark theme variations.

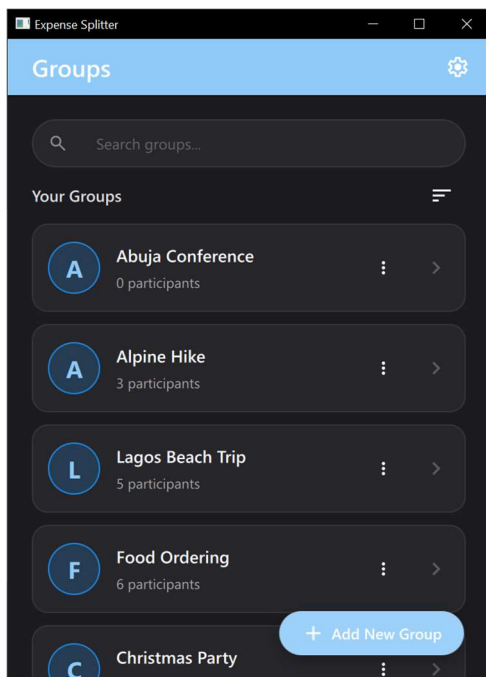


Fig. 4.1: Main interface displaying the trip list with search and sorting functionality.

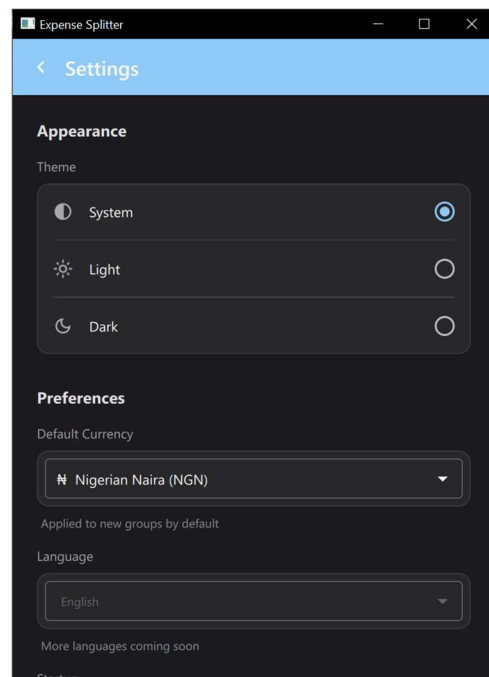


Fig. 4.2: Application settings page for theme and user preferences.

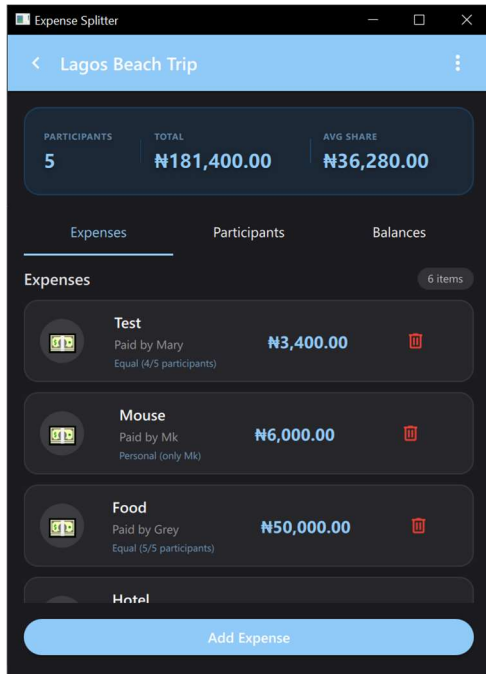


Fig. 4.3: Trip details page displaying recorded expenses (Expenses tab – default view).

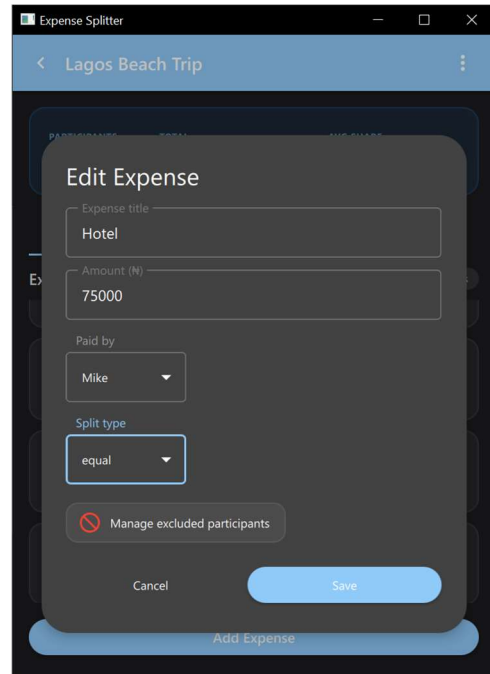


Fig. 4.4: Expense entry dialog used to add or edit an expense.

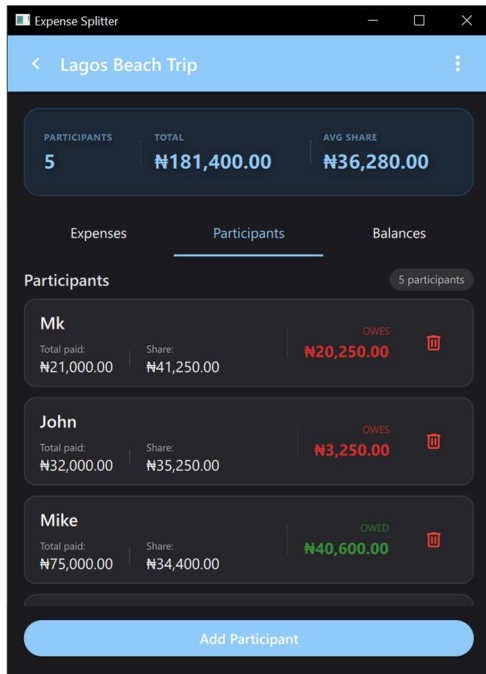


Fig. 4.5: Participants tab displaying group members and individual balances.

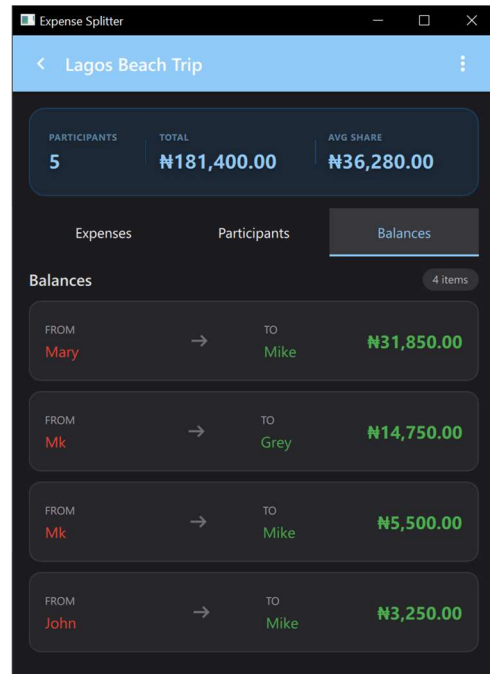


Fig. 4.6: Settlement tab presenting automated settlement recommendations.

Lagos Beach Trip
Created January 04, 2026 • 5 participants • NGN

Total Spent	Expenses	Avg / Person
181,400 NGN	6	36,280 NGN

Expenses Breakdown

Expense	Amount	Paid By	Split	Date
Test	3,400.00	Mary	Equal – excl. 1	Jan 14
Mouse	6,000.00	Mk	Personal	Jan 11
Food	50,000.00	Grey	Equal	Jan 11
Hotel	75,000.00	Mike	Equal	Jan 11
Transport	32,000.00	John	Equal	Jan 10
Fares	15,000.00	Mk	Equal	Jan 09

Participant Balances

Participant	Paid	Should Pay	Balance
Mike	75,000.00	34,400.00	+40,600.00
Grey	50,000.00	35,250.00	+14,750.00
John	32,000.00	35,250.00	-3,250.00
Mk	21,000.00	41,250.00	-20,250.00

Fig. 4.7: PDF summary generated for sharing trip expense records.

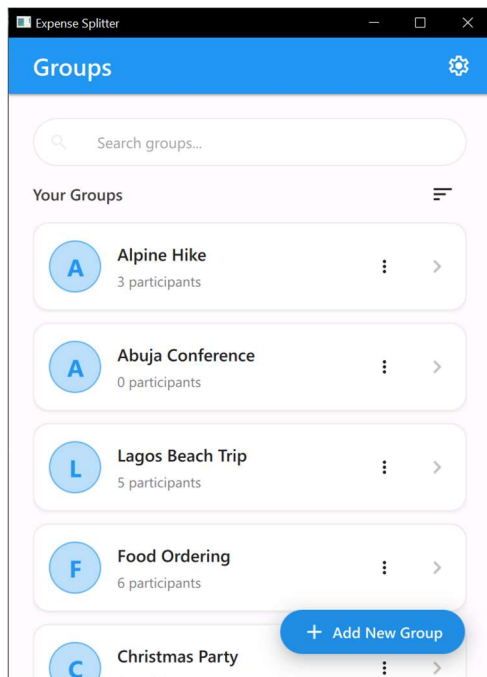


Fig. 4.8: Application interface displayed in light theme.

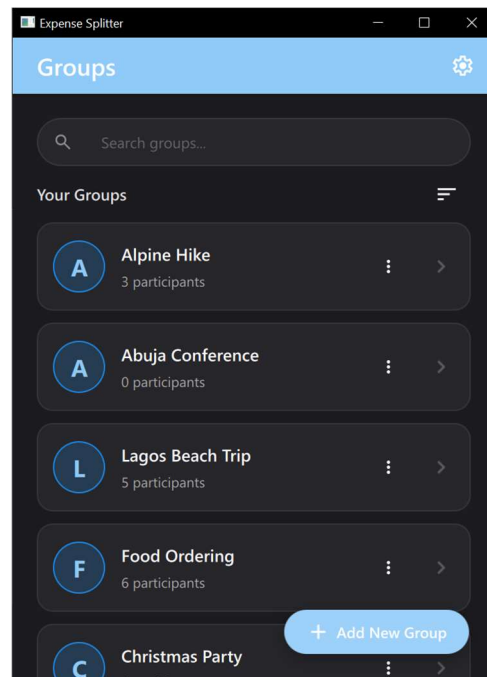


Fig. 4.9: Application interface displayed in dark theme.

4.2 Sample Input and Output

Sample Input:

- Trip name: *"Weekend Hangout"*
- Four participants are added: *A, B, C, D*
- Expenses recorded
 - ₦24,000 – Food (paid by *A*, split among all)
 - ₦10,000 – Transport (paid by *B*, excluding *D*)
 - ₦5,000 – Personal expense (paid by *C*)

Sample Output:

- Balances are calculated and displayed for each participant.
- Settlement suggestions are generated using these balances, for example:
 - *C* pays *A* ₦9,333.33
 - *D* pays *A* ₦5,333.34
 - *D* pays *B* ₦666.67

4.3 Key Code Snippets

Some of the code examples cover database initialization, expense calculations and settlement procedures. These snippets demonstrate the app's approach to processing and presenting data.

Only selected code snippets are shown to illustrate key functionality. The complete source code is available in *Appendix A*.

Database Initialization:

```
def _open_db(self):
    base = QStandardPaths.writableLocation(QStandardPaths.AppDataLocation)
    Path(base).mkdir(parents=True, exist_ok=True)
    db_path = Path(base) / "app.db"

    self.db = QSqlDatabase.addDatabase("SQLITE", "app_connection")
    self.db.setDatabaseName(str(db_path))

    if not self.db.open():
        raise RuntimeError("Failed to open SQLite database")

    query = QSqlQuery(self.db)

    # — Table Schemas —————
    query.exec("PRAGMA foreign_keys = ON")

    # Trips
    query.exec("""
        CREATE TABLE IF NOT EXISTS trips (
```

```

        id TEXT PRIMARY KEY,
        name TEXT NOT NULL,
        currency TEXT NOT NULL,
        created_at TEXT NOT NULL,
        updated_at TEXT NOT NULL
    );
    """
    # Participants
    query.exec("""
        CREATE TABLE IF NOT EXISTS participants (
            id TEXT PRIMARY KEY,
            trip_id TEXT NOT NULL,
            name TEXT NOT NULL,
            FOREIGN KEY (trip_id) REFERENCES trips(id) ON DELETE CASCADE
        );
    """)
    # Expenses
    query.exec("""
        CREATE TABLE IF NOT EXISTS expenses (
            id TEXT PRIMARY KEY,
            trip_id TEXT NOT NULL,
            title TEXT NOT NULL,
            amount REAL NOT NULL,
            paid_by TEXT NOT NULL,
            split_type TEXT NOT NULL,
            created_at TEXT NOT NULL,
            FOREIGN KEY (trip_id) REFERENCES trips(id) ON DELETE CASCADE
        );
    """)
    # Expense exclusions (many-to-many)
    query.exec("""
        CREATE TABLE IF NOT EXISTS expense_excluded (
            expense_id TEXT NOT NULL,
            participant_id TEXT NOT NULL,
            PRIMARY KEY (expense_id, participant_id),
            FOREIGN KEY (expense_id) REFERENCES expenses(id) ON DELETE CASCADE,
            FOREIGN KEY (participant_id) REFERENCES participants(id) ON DELETE CASCADE
        );
    """)

```

Expense splitting/ Balance Calculation:

```

def get_participant_balances(
    participants: List[Dict], expenses: List[Dict]
) -> Dict[str, Dict]:
    """Calculate the balance for each participant based on expenses"""
    participant_map = {p["id"]: p["name"] for p in participants}
    balances = {
        pid: {"name": name, "total_paid": 0.0, "should_pay": 0.0, "balance": 0.0}
        for pid, name in participant_map.items()
    }

    for expense in expenses:
        amount = expense.get("amount", 0.0)
        paid_by = expense.get("paid_by")

```



```

split_type = expense.get("split_type", "equal")
excluded = set(expense.get("excluded", []))

# Who participates in this expense?
participants = [pid for pid in participant_map if pid not in excluded]
participant_count = len(participants) or 1 # avoid division by zero

# Credit the payer (even if personal)
if paid_by and paid_by in balances:
    balances[paid_by]["total_paid"] += amount

if split_type == "personal":
    # Only the payer should pay (if known)
    if paid_by and paid_by in balances:
        balances[paid_by]["should_pay"] += amount
    if paid_by and paid_by not in balances:
        print(
            f"Warning: Unknown payer {paid_by} for expense {expense.get('id')}"
        )

else: # "equal" (default)
    share = round(amount / participant_count, 2)
    for mid in participants:
        if mid in balances:
            balances[mid]["should_pay"] += share

for data in balances.values():
    data["balance"] = data["total_paid"] - data["should_pay"]

return balances

```

Settlement algorithm (makes use of the in-built heapq library to quickly access highest debtors and creditors):

```

def get_settlement_transactions(balances: Dict[str, Dict]) -> List[Dict]:
    """Generate settlement transactions to balance accounts"""
    creditors = [] # max-heap → negative values
    debtors = [] # max-heap → negative values for abs(debt)
    TOLERANCE = 0.01

    for pid, data in balances.items():
        balance = round(data["balance"], 2)
        if balance > TOLERANCE:
            heapq.heappush(
                creditors, (-balance, data["name"], pid)
            ) # negative = max heap
        elif balance < -TOLERANCE:
            heapq.heappush(debtors, (balance, data["name"], pid)) # negative balance

    settlements = []

    while creditors and debtors:
        cred_amt_neg, cred_name, cred_id = heapq.heappop(creditors)
        debt_amt, debt_name, debt_id = heapq.heappop(debtors)

```

```

cred_amt = -cred_amt_neg
pay_amount = min(cred_amt, -debt_amt)

settlements.append({
    "from_id": debt_id,
    "from_name": debt_name,
    "to_id": cred_id,
    "to_name": cred_name,
    "amount": round(pay_amount, 2),
})

# Update amounts
cred_amt -= pay_amount
debt_amt += pay_amount # debt_amt becomes less negative

# Move to next creditor or debtor if settled
if cred_amt > 0.01:
    heapq.heappush(creditors, (-cred_amt, cred_name, cred_id))
if debt_amt < -0.01:
    heapq.heappush(debtors, (debt_amt, debt_name, debt_id))

return settlements

```

4.4 Performance Evaluation

The application generally performs well, with smooth navigation and near-instant calculations. A small delay occurs during startup due to Qt loading and database initialization, and this is being optimized. Memory usage stays within a reasonable range, typically around 80–150 MB depending on data size.

4.5 User Testing Results

Five users participated in testing the application. Feedback highlighted the intuitive interface, accurate calculations, and positive overall experience. Settlement suggestions and the ability to switch between themes were especially appreciated.

Tester	User Type	Task Performed	Feedback
User 1	Student	Create trip, add expenses	Interface was intuitive
User 2	Student	Add participants, view balances	Calculations were clear
User 3	Student	Generate PDF	Export was useful
User 4	Student	Switch themes	Dark mode improved readability
User 5	Student	View settlements	Settlement logic easy to understand

Chapter 5: Conclusion and Recommendations

5.1 Summary of Achievements

We built a complete offline Expense Splitter with intuitive mobile-inspired GUI, reliable SQLite storage, real-time balances, smart settlement suggestions, themes, currency support, search, sort, and PDF sharing, meeting all objectives.

5.2 Conclusion

This project proves that Python with PySide6 and QML can create attractive and functional desktop apps that can solve real-world problems.

5.3 Challenges and Lessons Learned

Throughout the development process, we encountered some challenges and learned some lessons:

- Implementing UI consistency across themes was challenging (especially getting dark mode to work properly after ThemeManager changes).
- Switching from JSON blob in registry to SQLite mid-project meant rewriting storage logic, but it improved everything.
- Learned better Qt patterns, importance of early prototyping, and why good version control saves headaches.

5.4 Future Improvements

For future enhancements, we considered some possibilities including:

- Adding percentage/custom/unequal splits and multiple payers.
- Creating a mobile version (design is already mobile oriented).
- Make current design more desktop friendly.
- Interactive charts and visualizations.
- Better security (e.g., encryption for sensitive trips).
- Cloud sync option.

5.5 Contribution and Usefulness

It's a free, offline tool for anyone splitting costs, such as students on trips, roommates, or friends, and it contributes a clean Python/Qt example for desktop apps while showing the full software development cycle from idea to GitHub.

References

- [1] Python Software Foundation, "Python Documentation," [Online]. Available: <https://docs.python.org/3/>. Accessed: Feb. 6, 2026.
- [2] Qt Company, "Qt for Python (PySide6) Documentation," [Online]. Available: <https://doc.qt.io/qtforpython/>. Accessed: Feb. 6, 2026.
- [3] SQLite Consortium, "SQLite Documentation," [Online]. Available: <https://www.sqlite.org/docs.html>. Accessed: Feb. 6, 2026.
- [4] ReportLab, "ReportLab User Guide," [Online]. Available: <https://www.reportlab.com/docs/reportlab-userguide.pdf>. Accessed: Feb. 6, 2026.
- [5] Google, "Material Design Guidelines," [Online]. Available: <https://material.io/design>. Accessed: Feb. 6, 2026.

Appendices

Appendix A: Project Dependencies (requirements.txt)

Appendix B: User Manual

Appendix C: GitHub Repository

<https://github.com/Mk9397/Expense-Splitter>

Appendices

Appendix A: Project Dependencies (requirements.txt)

The following Python packages are required to run the application. Install them using:

```
pip install -r requirements.txt
```

Content of requirements.txt:

```
PySide6==6.10.1
```

```
reportlab==4.2.2
```

Notes:

- PySide6 provides the Qt bindings for GUI (including QML support).
- ReportLab is used solely for generating PDF summaries.
- SQLite is built-in via QSql — no extra package is needed.
- Python version: 3.12 or higher recommended.
- No additional heavy dependencies.

Appendix B: User Manual

Bells University Expense Splitter

Version 1.0

Offline Desktop Application for Group Expense Management

Overview

This standalone application helps groups (e.g., students on trips, roommates, friends) track shared expenses, calculate balances instantly, suggest fair settlements with minimal payments, and export reports, all without internet.

System Requirements

- Windows/Linux/macOS desktop
- Python 3.12+ (for source run) or the standalone .exe (Windows)
- Typical usage of ~150 MB RAM

Installation

Source Code

1. Clone the repository: git clone <https://github.com/Mk9397/Expense-Splitter.git>
2. Navigate to the project folder: cd Expense-Splitter
3. Install dependencies: pip install -r requirements.txt
4. Run the application: python main.py

Standalone Executable

An .exe file for Windows (no Python install needed) is available on the GitHub repository. Download, double-click to run — no installation required.

Getting Started

1. On launch, the **Homepage** shows existing trips (as cards) with a search bar and sort options.
2. Click the **Floating Action Button** (+) to create a new trip:
 - Enter trip name (e.g., "Weekend Hangout")
 - Select currency (default: ₺)
3. Add participants: Go to **Participants** tab → Add names (e.g., A, B, C, D).

Adding Expenses

1. In a trip, go to **Expenses** tab (default view).
2. Click **Add Expense**:

- Title (e.g., "Food")
 - Amount (e.g., 24000)
 - Paid by: Select participant
 - Split type: Equal (default) or Personal
 - Exclude participants if needed (checkboxes)
3. Save → Balances update instantly across tabs.

Viewing Balances & Settlements

- **Participants** tab: Shows each person's total paid, should pay, and net balance.
- **Settlements** tab: Displays optimal payment chain (e.g., "C pays A ₦9,333.33", "D pays B ₦666.67") using minimal transactions. Balances refresh in real-time after every expense.

Additional Features

- **Themes:** Go to Settings → Choose Light, Dark, or System (matches OS).
- **Export PDF:** In any trip, click Export → Generates a summary (expenses, balances, settlements) saved to Documents folder.
- **Search/Sort:** Filter trips by name; sort by creation date or name.
- Data saved automatically in local SQLite file (no cloud/sync).

Tips & Troubleshooting

- Personal expenses: Mark as "personal" so only payer is charged.
- Exclusions: Use for items not shared by all (e.g., transport excluding one person).
- Floating balances < ₦0.01 ignored for settlements.
- If app fails to start: Ensure PySide6 is installed and check console for errors.
- For issues: Report on GitHub Issues.

Support

Open issues at: <https://github.com/Mk9397/Expense-Splitter/issues>

Appendix C: GitHub Repository

Repository URL: <https://github.com/Mk9397/Expense-Splitter>

Description:

An offline Python desktop application for splitting group expenses, built with PySide6 (Qt), QML UI, SQLite storage, and ReportLab for PDF exports. Features real-time balances, optimal settlements, themes, and more.

Key Contents:

- Source code in src/ and qml/
- Screenshots in screenshots/ folder
- requirements.txt for dependencies
- Full project documentation and report references

Access:

Visit the link above to view code, download, or contribute. A Windows executable (exe) is available in the **Releases** section for easy one-click use without Python installation.